NIXL Cluster Topology Discovery

Problem statement

Goals

1. **Measure/characterize inter-node memory transfer performance**
2. **Infer node connectivity graph**

Constraints

- Must use NIXL memory operations for test setup and execution
- Multi-process implementation even on a single node

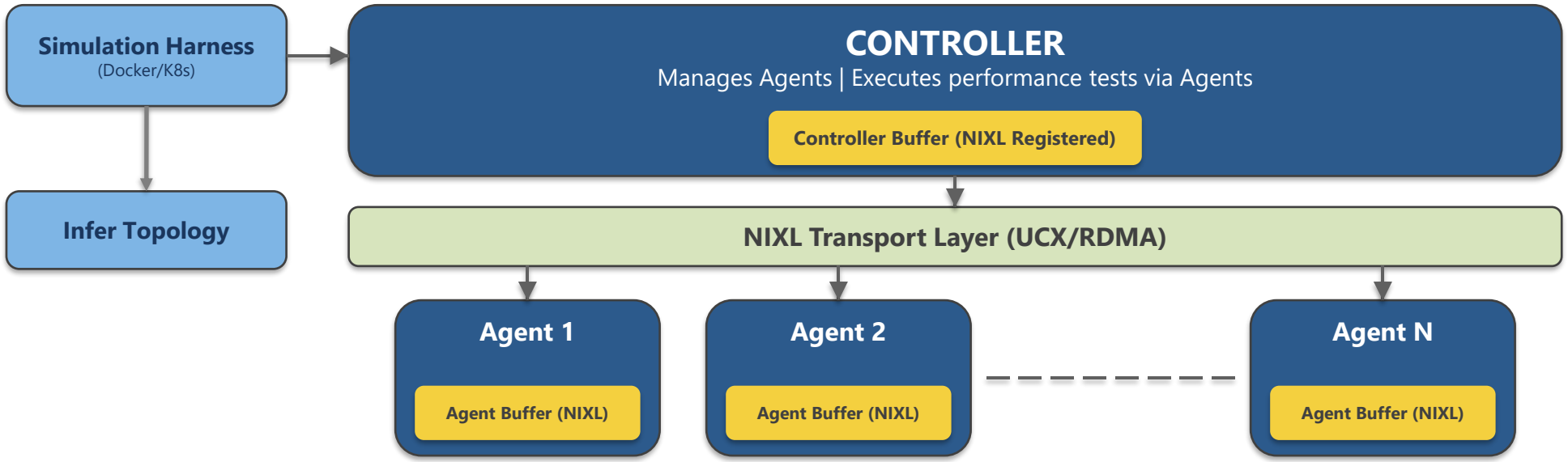
- Introduce hidden nodes to represent system bottlenecks
- Visualize connectivity graph
- Simulation environment with ability to configure topology and link capacities

Outline

1. System implementation (WHAT)
   - Architecture components
   - Module communication
   - Memory performance tests
   - Inferring cluster topology
   - Simulation

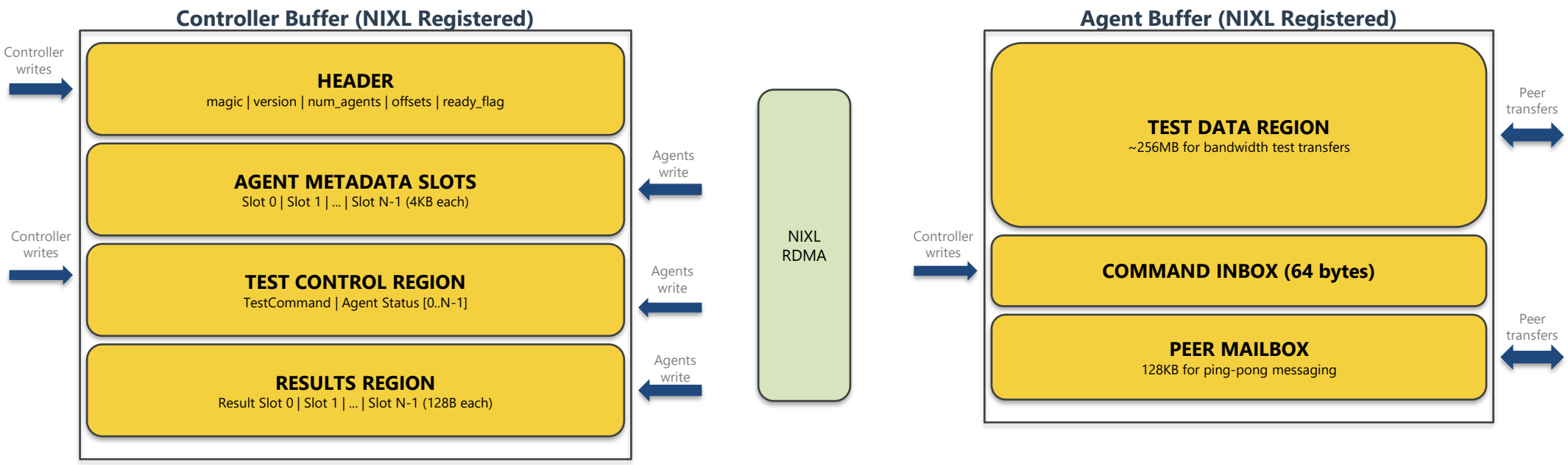
2. AI usage (HOW)
   - Overall development workflow
   - Share prompts and experience

System Architecture - Component Overview



- Controller and agents run as containers spawned via Docker/Kubernetes
- All IPC via NIXL RDMA using memory semantics - no TCP sockets or other IPC mechanisms
- Topology inferred via clustering algorithm using collected memory performance metrics
- Simulation Harness kicks off Controller, creates cluster network topology, collects results

Controller - Agent Communication



- All transfers use NIXL memory semantics - zero-copy, kernel-bypass RDMA
- Controller Buffer used to exchange address, command info and results
- Every buffer region has only one writer.
- Controller and Agents always poll local memory, not network
- Peer-to-peer tests use agent mailbox/data regions - controller not in data path

Memory Performance Tests

Goal: Evaluate memory transfer performance between cluster nodes

Latency

Round-trip time for small messages

Ping-Pong Test

- Two nodes exchange small messages (8 bytes)
- Measures round-trip time (RTT) / 2 for 1-way latency
- Reveals base communication overhead

Insight: How far apart are these nodes? (topology distance)

Throughput

Maximum sustainable data transfer rate

Bandwidth Sweep Test

- Sender streams large messages, receiver acks
- Tests multiple msg sizes to find peak throughput
- Identifies optimal message size for max transfer rate

Insight: What is the peak throughput between nodes? (capacity)

Transfer Time

How latency scales with message size

Latency Sweep Test

- Runs ping-pong across message sizes (8B to 64KB)
- Plots transfer time vs size
- Finds bandwidth saturation point

Insight: What kind of link connects the nodes? (characterization)

Note: These tests are based on [OSU Micro-Benchmarks (OMB)]() memory performance test methodology.

Topology Generation

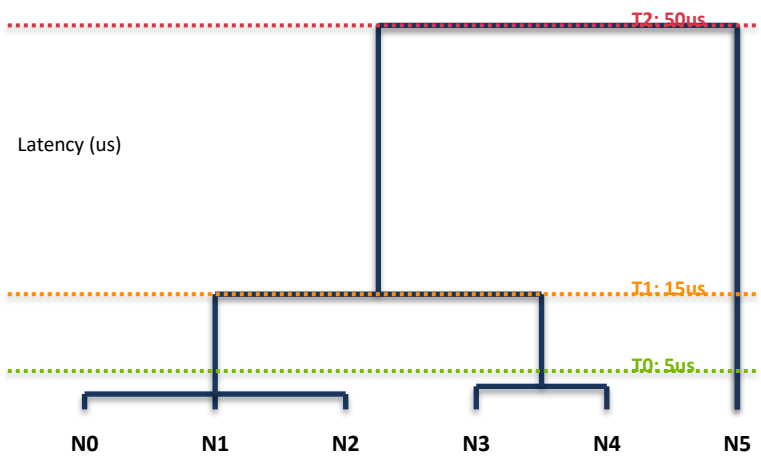
Infers connectivity graph using collected latency and bandwidth data

Input: Latency Matrix (us)

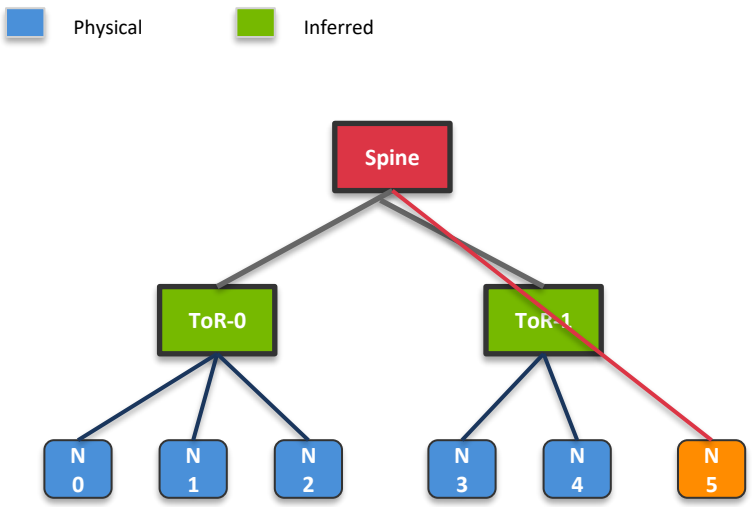
	N0	N1	N2	N3	N4	N5
N0	0	2	2	15	15	50
N1	2	0	2	15	15	50
N2	2	2	0	15	15	50
N3	15	15	15	0	3	50
N4	15	15	15	3	0	50
N5	50	50	50	50	50	0

2-3us: Same rack 15us: Cross rack 50us: Remote

Dendrogram (Hierarchical Clustering)



Output: Inferred Topology



Algorithm Details

1) Clustering Algorithm

a) Iteratively merge node pairs with the lowest latency

b) Merged node "metric" = latency between merged nodes

c) Calculate merged node's latency to remaining nodes (min/max/weighted average of constituent nodes)

d) Generate a Dendrogram (tree) that records merge history

2) Tier Thresholds

Represent network element latencies:

- Links, switches, unknown bottlenecks

Simulation uses tc network params to derive thresholds automatically (geometric mean of tc delays)

3) Hidden Node Generation

a) Depth First Traversal from dendrogram root

b) If metric >= threshold: add hidden node, add edges to children

If metric < threshold: group children together

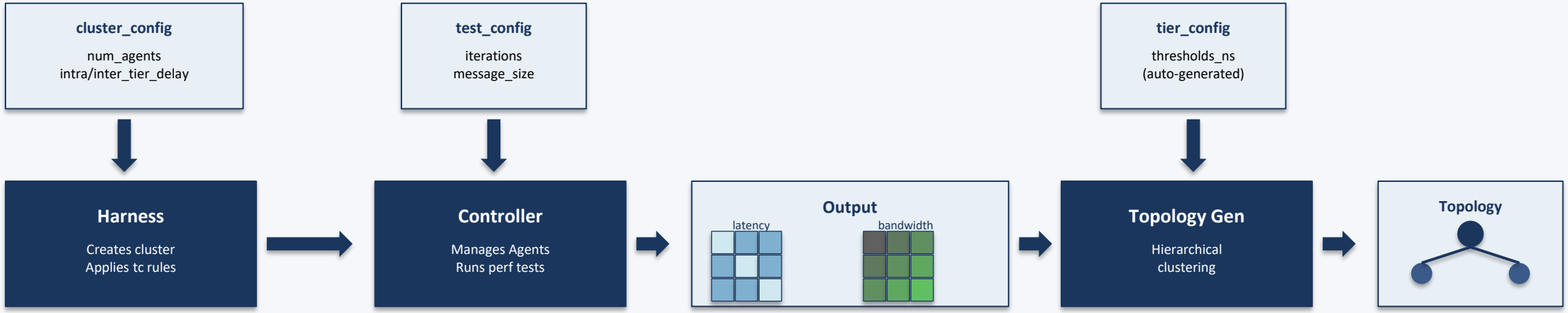
c) Label edges with link capacity

(phase 2: peak measured bandwidth)

Cluster Topology Simulation

Simulate cluster topologies to test discovery algorithms without physical multi-rack infrastructure

Simulation Flow



Implementation Details

- Controller and Agents run in containers
- Harness configures cluster topology
- Harness calls tc to set link latencies (HTB qdisc algorithm)
- Volume mount to share config and result files with controller
- Harness invokes topo-gen to visualize inferred topology (dot, png, svg)

Auto-Generated Tier Thresholds

Harness computes optimal tier thresholds from cluster_config.

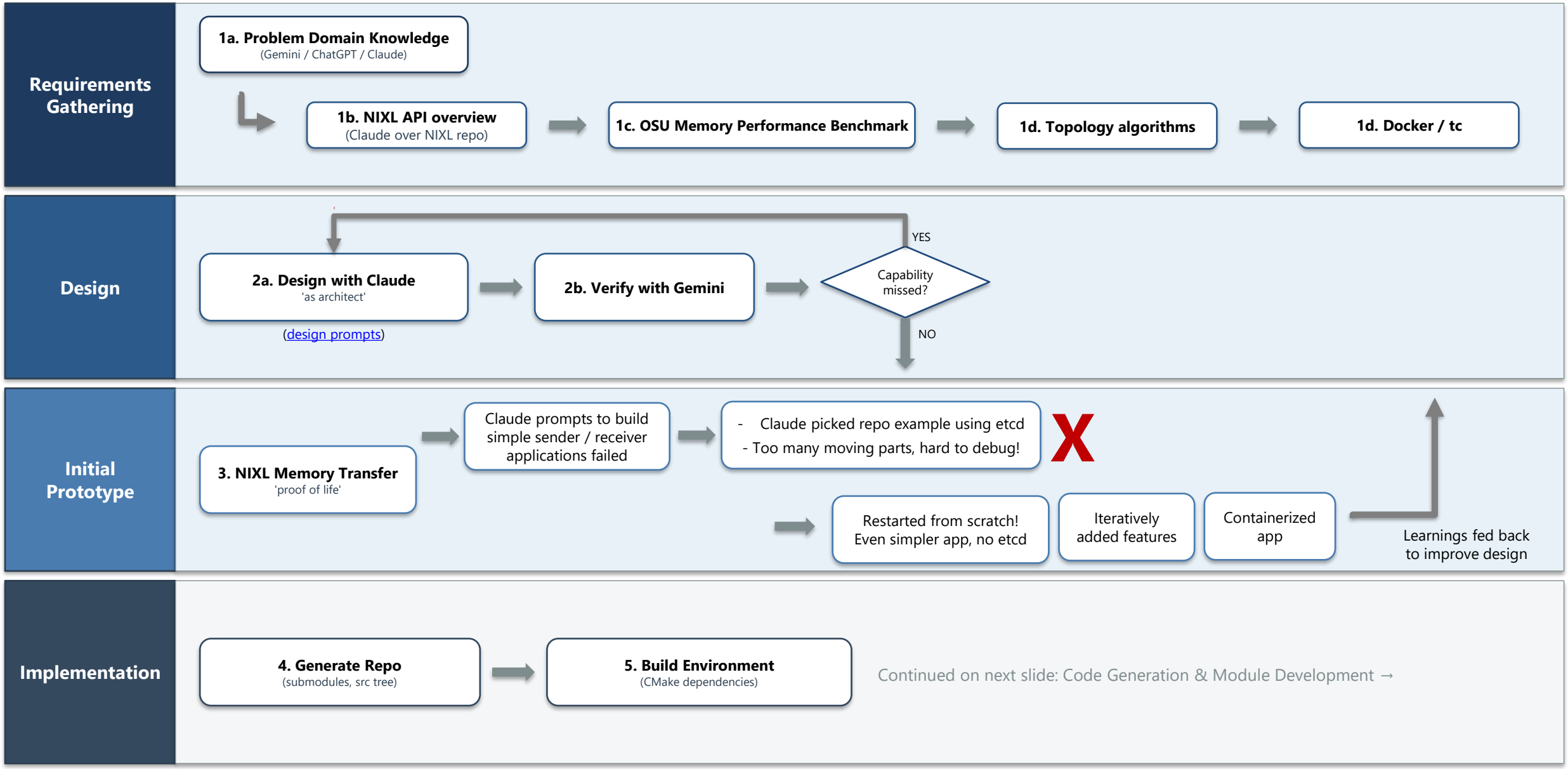
Formula: threshold = √(intra_tier × inter_tier)

Provides log-scale separation for hierarchical clustering.

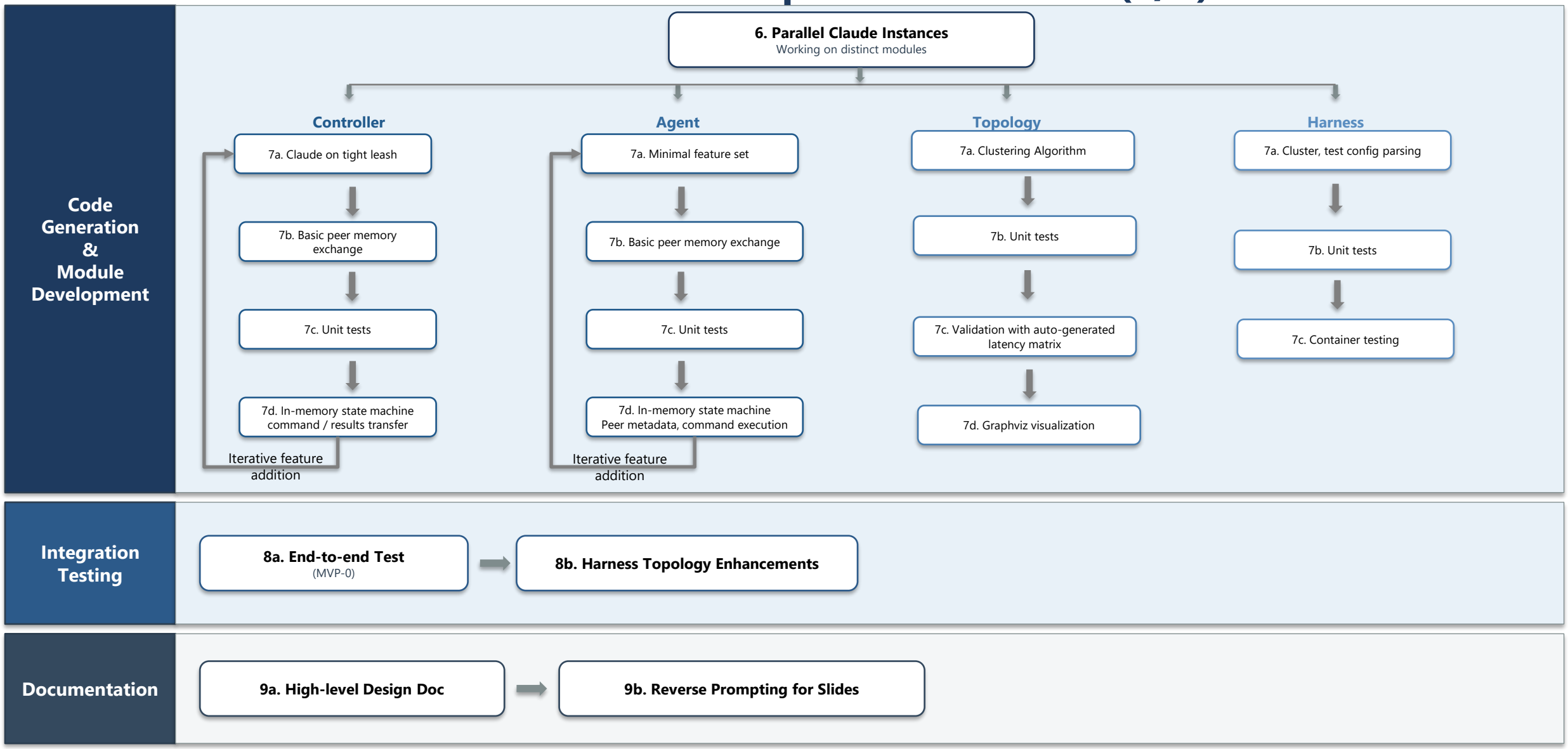
Example:

intra: 100µs, inter: 5000µs → √(100×5000) = 707µs

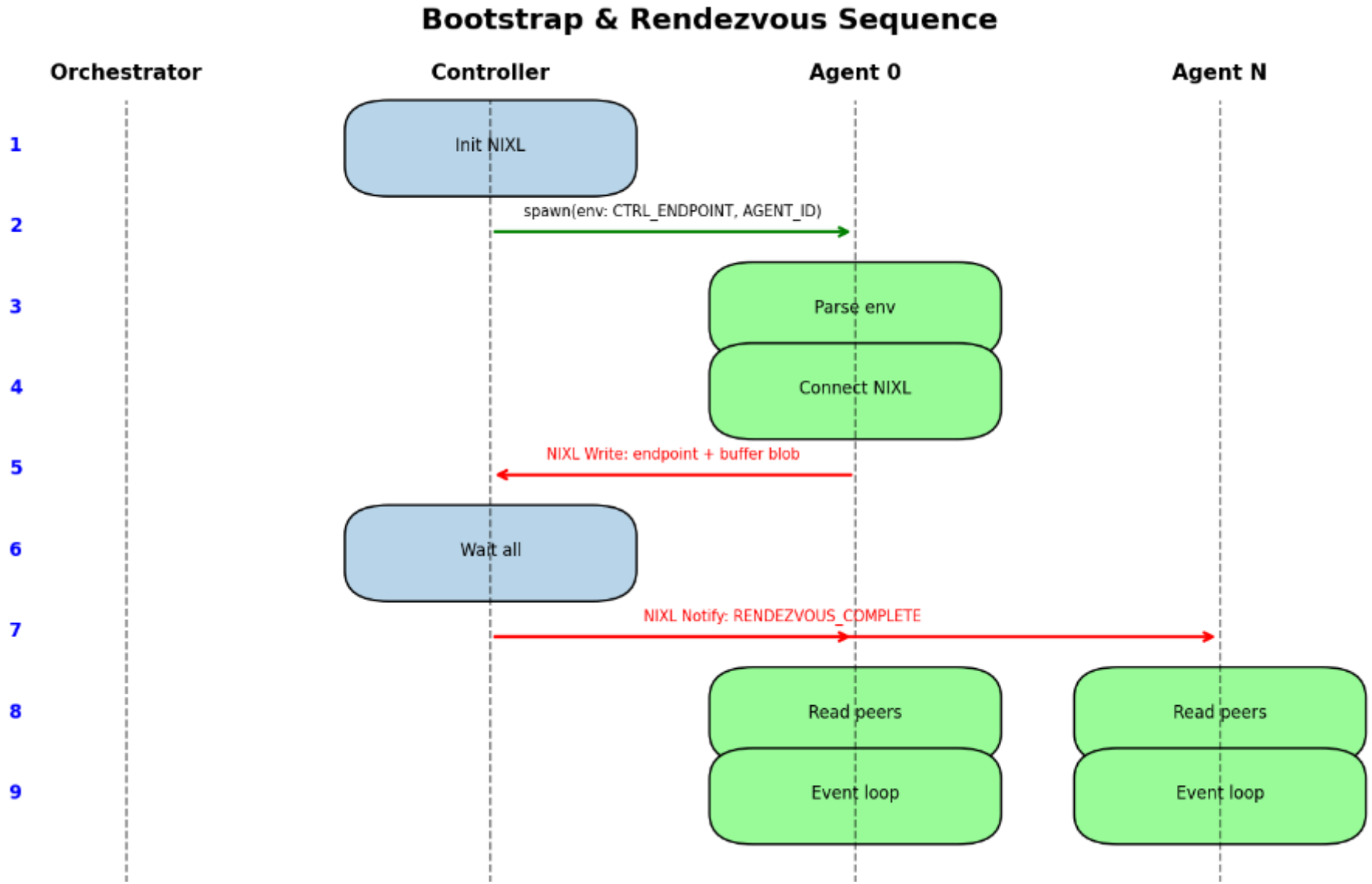
AI Assistance - Development Workflow (1/2)

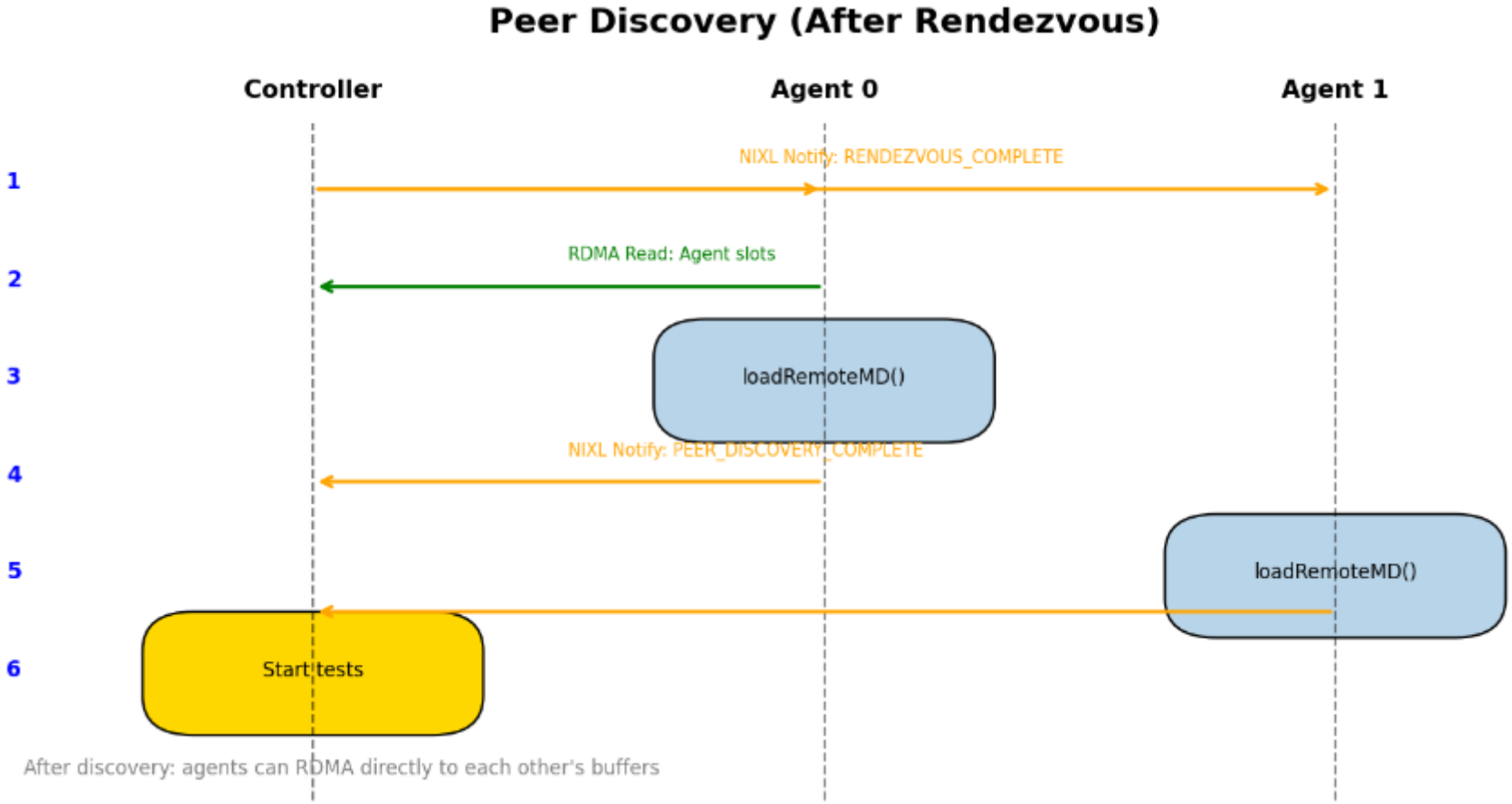


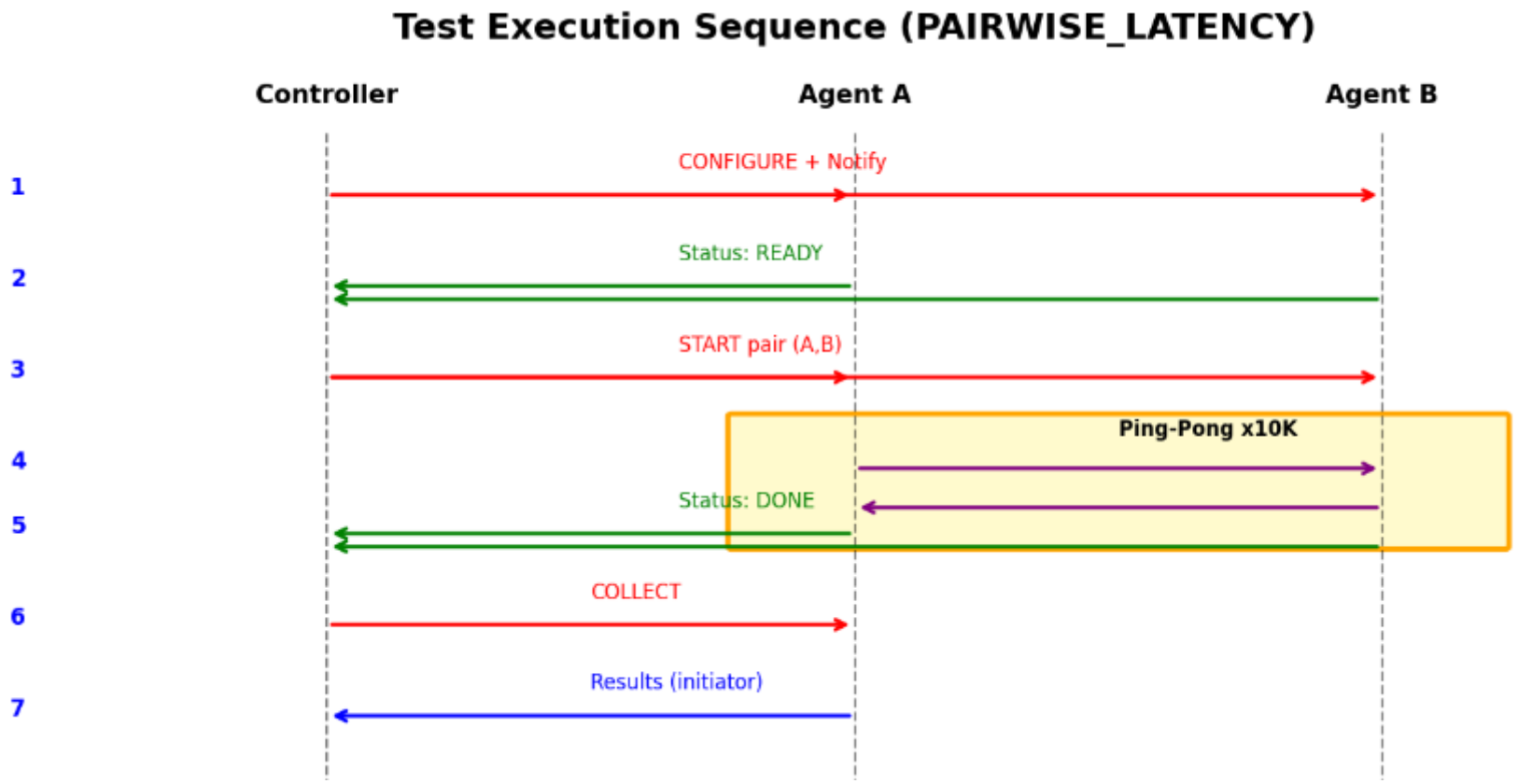
AI Assistance - Development Workflow (2/2)

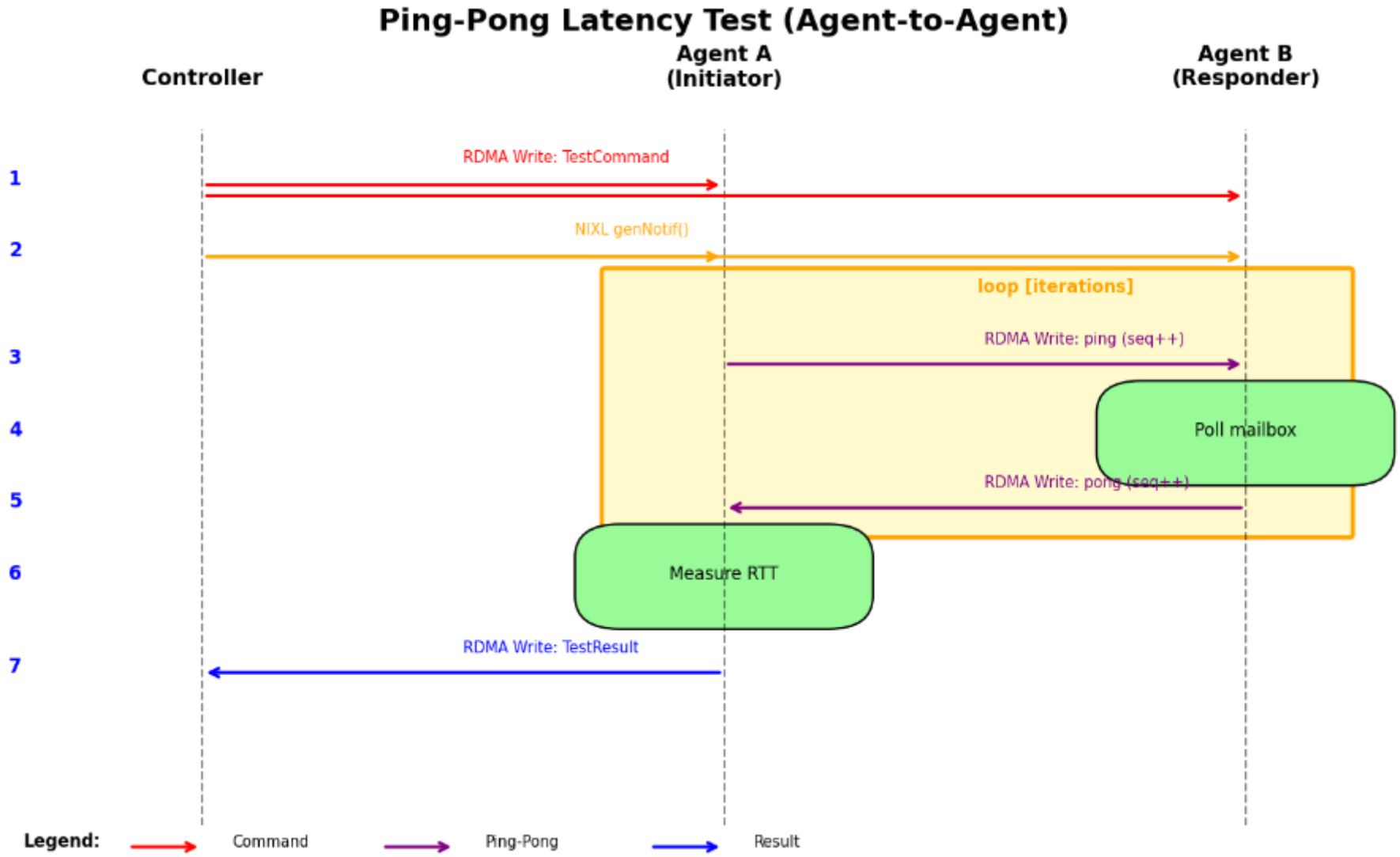


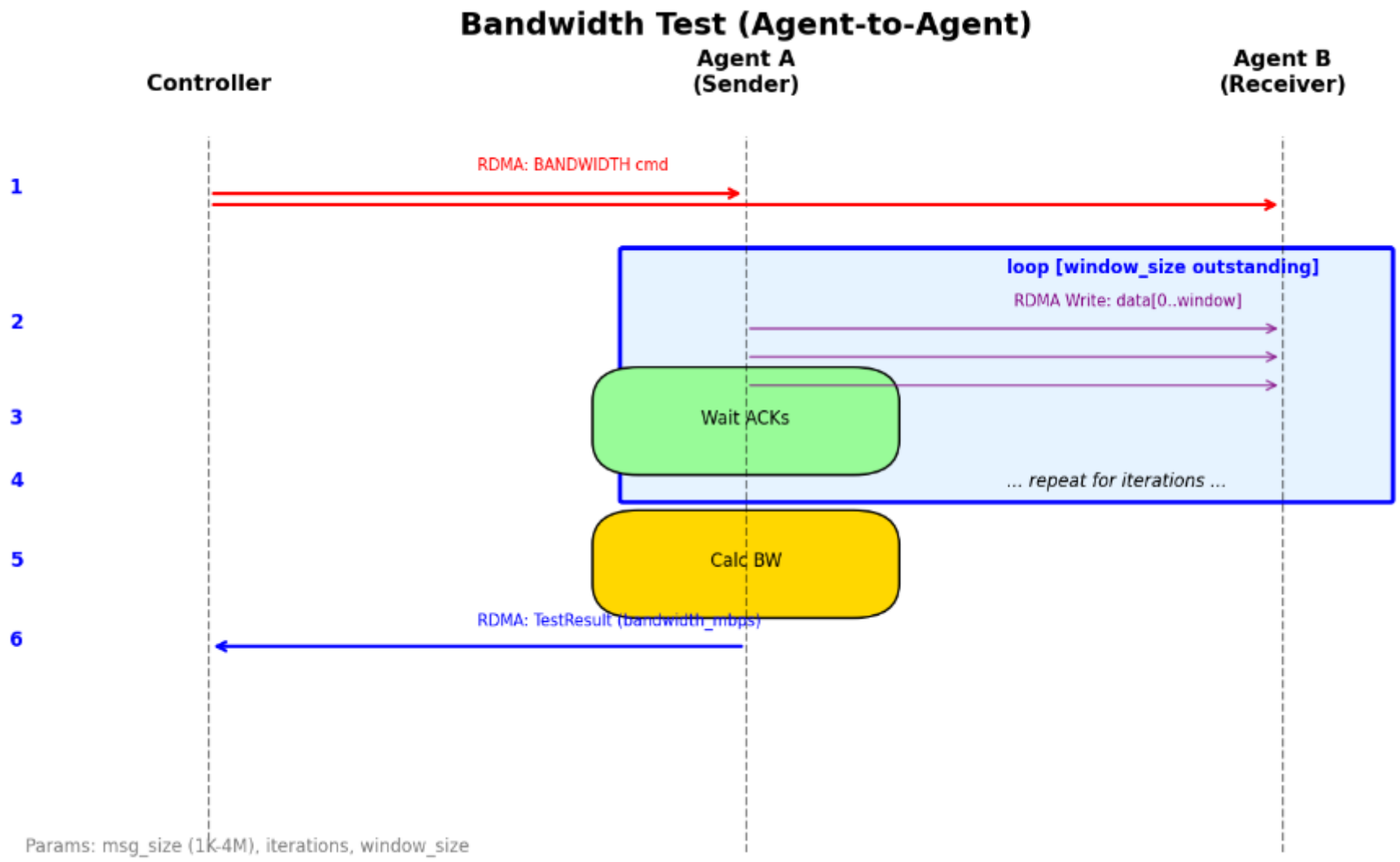
Backup


Bootstrap & Rendezvous Sequence
Orchestrator
Controller
Agent 0
Agent N
1
Init NIXL
2
spawn(env: CTRL_ENDPOINT, AGENT_ID)
3
Parse env
4
Connect NIXL
5
NIXL Write: endpoint + buffer blob
6
Wait all
7
NIXL Notify: RENDEZVOUS_COMPLETE
8
Read peers
Read peers
9
Event loop
Event loop


Peer Discovery (After Rendezvous)
Controller
Agent 0
Agent 1
1
NIXL Notify: RENDEZVOUS_COMPLETE
2
RDMA Read: Agent slots
3
loadRemoteMD()
4
NIXL Notify: PEER_DISCOVERY_COMPLETE
5
loadRemoteMD()
6
Start tests
After discovery: agents can RDMA directly to each other's buffers


Test Execution Sequence (PAIRWISE_LATENCY)
Controller
Agent A
Agent B
1
CONFIGURE + Notify
2
Status: READY
3
START pair (A,B)
4
Ping-Pong x10K
5
Status: DONE
6
COLLECT
7
Results (initiator)


Ping-Pong Latency Test (Agent-to-Agent)
Controller
Agent A
(Initiator)
Agent B
(Responder)
1
RDMA Write: TestCommand
2
NIXL genNotif()
loop [iterations]
3
RDMA Write: ping (seq++)
4
Poll mailbox
5
RDMA Write: pong (seq++)
6
Measure RTT
7
RDMA Write: TestResult
Legend:
Command
Ping-Pong
Result


Bandwidth Test (Agent-to-Agent)
Controller
Agent A
(Sender)
Agent B
(Receiver)
1
RDMA: BANDWIDTH cmd
loop [window_size outstanding]
2
RDMA Write: data[0..window]
3
Wait ACKs
4
... repeat for iterations ...
5
Calc BW
6
RDMA: TestResult (bandwidth_mbps)
Params: msg_size (1K-4M), iterations, window_size